RK801 开发指南

文件标识：RK-KF-YF-C57

发布版本：V1.1.0

日期： 2025-11-12

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档主要介绍 RK801 的各个子模块，介绍相关概念、功能、dts 配置和一些常见问题的分析定位。

产品版本

芯片名称	内核版本
RK801	6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	陈健洪	2025-10-23	初始版本
V1.1.0	陈健洪	2025-11-12	更新soc_pwrctrl_reset引脚配置说明

目录

RK801 开发指南

1. 基础
   - 1.1 概述
   - 1.2 芯片功能
   - 1.3 芯片引脚
   - 1.4 重要概念
   - 1.5 上电条件和时序
2. 配置
   - 2.1 驱动和 menuconfig
   - 2.2 驱动版本
   - 2.3 DTS 配置
   - 2.4 函数接口
3. 调试

1. 基础

1.1 概述

RK801 是一款高性能 PMIC，RK801 集成 4 个大电流 DCDC、2 个 LDO、1 个开关SWITCH、1 个PWRON、可调上电时序等功能。

系统中各路电源总体分为两种：DCDC 和 LDO。两种电源的总体特性如下（详细资料请自行搜索）：

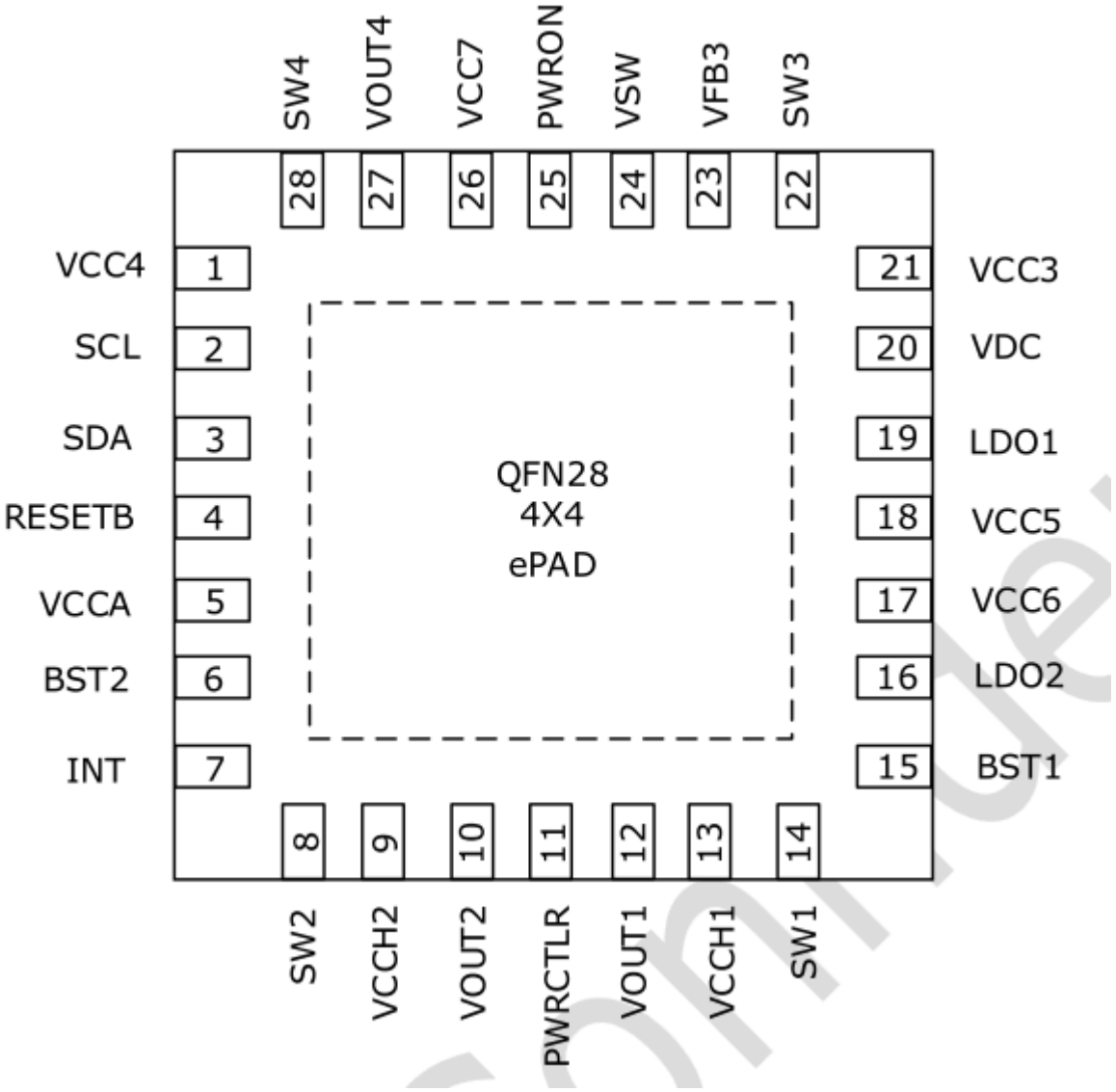
1. DCDC：输入输出压差大时，效率高，但是存在纹波比较大的问题，成本高，所以大压差，大电流负载时使用。一般有两种工作模式。PWM 模式：纹波瞬态响应好，效率低；PFM 模式：效率高，但是负载能力差。
2. LDO：输入输出压差大时，效率低，成本低，为了提高 LDO 的转换效率，系统上会进行相关优化如：LDO 输出电压为 1.1V，为了提高效率，其输入电压可以从 VCCIO_3.3V 的 DCDC 给出。所以电路上如果允许尽量将 LDO 接到 DCDC 输出回路，但是要注意上电时序。

1.2 芯片功能

从使用者的角度看，RK801 的功能概况起来可以分为 2 个部分：

1. regulator 功能：控制各路 DCDC、LDO 电源状态；
2. pwrkey 功能：检测 power 按键的按下/释放，可以为 AP 节省一个 gpio。

1.3 芯片引脚



下面描述中，PWRCTRL 和 INT 引脚需要重点关注：

PIN NO	PIN NAME	PIN DESCRIPTION
1	VCC4	Power supply of BUCK4
2	SCL	I2C clock input(Open drain output)
3	SDA	I2C data input and output(Open drain output)
4	RESETB	Reset pin after power on, active low;
5	VCCA	Power supply of inner circuit;
6	BST2	Boostrap capacitor of High voltage BUCK2
7	INT	Interrupt request pin(Open drain output)
8	SW2	Switching node of High voltage BUCK2
9	VCCH2	Power supply of High voltage BUCK2
10	VOUT2	Output feedback voltage of high voltage of BUCK2
11	PWRCTLR	PWRCTLR control input.
12	VOUT1	Output feedback voltage of high voltage of BUCK1
13	VCCH1	Power supply of High voltage BUCK1
14	SW1	Switching node of High voltage BUCK1
15	BST1	Boostrap capacitor of High voltage BUCK1
16	LDO2	LDO2 output
17	VCC6	Power supply of LDO2
18	VCC5	Power supply of LDO1
19	LDO1	LDO1 output

PIN NO	PIN NAME	PIN DESCRIPTION
20	VDC	VDC input;
21	VCC3	Power supply of BUCK3
22	SW3	Switching node of BUCK3
23	VFB3	Output feedback voltage of BUCK3
24	VSW	Output of SWITCH
25	PWRON	Power on key input, active low, internal 22k resistor pull high to VCCA
26	VCC7	Power supply of SWITCH
27	VOUT4	Output feedback voltage of BUCK4
28	SW4	Switching node of BUCK4
Exposed pad	Exposed ground	Ground

1.4 重要概念

- I2C 地址

  7 位从机地址：0x27

- PMIC 有 3 种工作模式

  1. PMIC normal 模式

     系统正常运行时 PMIC 处于 normal 模式，此时 pmic_pwrctrl 为低电平。

  2. PMIC sleep 模式

     系统休眠时需要待机功耗尽量低，PMIC 会切到 sleep 模式减低自身功耗，这时候一般会降低某些路的输出电压，或者直接关闭输出，这可以根据实际产品需求进行配置。系统待机时 AP 通过 I2C 指令把 pmic_pwrctrl 配置成 sleep 模式，然后拉高 pmic_pwrctrl 即可让 PMIC 进入 sleep 状态；当 SoC 唤醒时 pmic_pwrctrl 恢复为低电平，PMIC 退出休眠模式。

3. PMIC shutdown 模式

   当系统进入关机流程的时候，PMIC 需要完成整个系统的电源下电操作。AP 通过 I2C 指令把 pmic_pwrctrl 配置成 shutdown 模式，然后拉高 pmic_pwrctrl 即可让 PMIC 进入 shutdown 状态。

目前驱动会根据dts配置和场景自动切换上述的各种模式，用户可以不关心。

- pmic_pwrctrl 引脚

  常态为低电平，PMIC 处于 normal 模式。当引脚拉高的时候会切换到 sleep 或者 shutdown 的模式。

- pmic_int 引脚

  常态为高电平，当有中断产生的时候变为低电平。如果中断没有被处理，则会一直维持低电平。

- pmic_pwron 引脚

  pwrkey 的功能需要硬件上将 power 按键接到这个引脚，驱动通过这个引脚来判断按下/释放。

- 各路 DCDC 的工作模式

  DCDC 有 PWM（也叫 force PWM）、PFM 模式，但是 PMIC 有一种模式会动态切换 PWM、PFM，这就是我们通常所说的 AUTO 模式。PMIC 支持 PWM、AUTO PWM/PFM 两种模式，AUTO 模式效率高但是纹波瞬态响应会差。出于系统稳定性考虑，运行时都是设置为 PWM 模式，系统进入休眠时会选择切换到 AUTO PWM/PFM。

- DCDC3 电压调节

  DCDC3 这路电源比较特殊，不能通过寄存器修改电压，只能通过外部电路的分压电阻进行调节，所以如果需要修改电压请修改外围硬件，在 Rockchip 的方案上一般作为 VCC_DDR 使用。

- DCDC 和 LDO 的运行时电压调节范围

  DCDC 电压范围不连续（见注释或datasheet）：

```
// BUCK1
static const struct linear_range rk801_buck1_voltage_ranges[] = {
   REGULATOR_LINEAR_RANGE(500000,   0, 80, 12500), /* 0.5v - 1.5v, 步进12.5mV */
   REGULATOR_LINEAR_RANGE(1800000, 81, 82, 400000),/* 1.8v - 2.2v */
   REGULATOR_LINEAR_RANGE(3300000, 83, 83, 0), /* 3.3v */
   REGULATOR_LINEAR_RANGE(5000000, 84, 84, 0), /* 5.0v */
   REGULATOR_LINEAR_RANGE(5250000, 85, 85, 0), /* 5.25v */
};

// BUCK2
static const struct linear_range rk801_buck2_voltage_ranges[] = {
   REGULATOR_LINEAR_RANGE(800000,  0, 2, 50000),   /* 0.8v - 0.9v */
   REGULATOR_LINEAR_RANGE(1800000, 3, 4, 400000),  /* 1.8v - 2.2v */
   REGULATOR_LINEAR_RANGE(3300000, 5, 5, 0),   /* 3.3v */
   REGULATOR_LINEAR_RANGE(5000000, 6, 6, 0),   /* 5.0v */
   REGULATOR_LINEAR_RANGE(5250000, 7, 7, 0),   /* 5.25v */
};

// BUCK4
static const struct linear_range rk801_buck4_voltage_ranges[] = {
   REGULATOR_LINEAR_RANGE(500000,   0, 80, 12500), /* 0.5v - 1.5v */
   REGULATOR_LINEAR_RANGE(1800000, 81, 82, 400000),/* 1.8v - 2.2v */
   REGULATOR_LINEAR_RANGE(2500000, 83, 83, 0), /* 2.5v */
   REGULATOR_LINEAR_RANGE(2800000, 84, 84, 0), /* 2.8v */
   REGULATOR_LINEAR_RANGE(3000000, 85, 85, 0), /* 3.0v */
```

```
    REGULATOR_LINEAR_RANGE(3300000, 86, 86, 0), /* 3.3v */
};
```

LDO 电压连续：

电压范围(V)	步进值(mV)
0.5 ~ 3.4	50

1.5 上电条件和时序

1. 上电条件

   只要满足下面任意一个条件即可以实现 PMIC 上电：

   - VDC 引脚为高电平
   - 下电状态下按 PWRON 键触发

2. 上电时序

   每款 SOC 平台对各路电源上电时序要求可能不一样，目前上电时序有如下情况，具体请参考最新的 datasheet。

2. 配置

2.1 驱动和 menuconfig

驱动文件：

```
drivers/mfd/rk808.c
drivers/regulator/rk801-regulator.c
drivers/input/misc/rk805-pwrkey.c
```

menuconfig 配置：

```
CONFIG_MFD_RK808
CONFIG_REGULATOR_RK801
CONFIG_INPUT_RK805_PWRKEY
```

2.2 驱动版本

请至少更新到如下版本确保驱动功能的完整性和稳定性。

drivers/mfd/rk808.c：

```
commit 0439f30ead1393813d69a01587530652908c9171
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Fri Jun 13 17:19:25 2025 +0800

    mfd: rk808: Print full chip id of rk801

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: I195858d23c862d0f961acb4c04bb1102ab9954c9
```

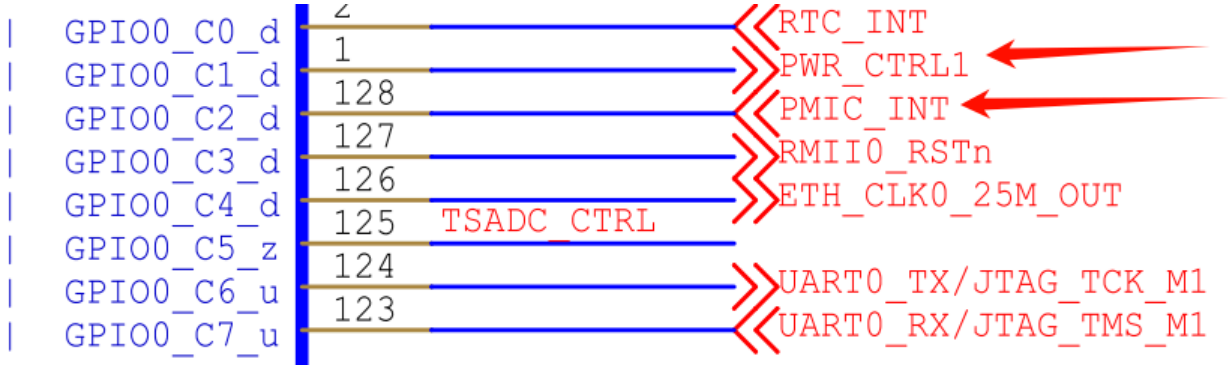
drivers/regulator/rk801-regulator.c：

```
commit 1a81f4f6344f7cd4203be508150daeedc00fbedb
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Thu May 29 11:39:58 2025 +0800

    mfd: rk801: Add support for pwrctrl active pol set and reboot rst

    - Don't require pwrctrl dvs if hw compatible version >= 3.
    - Auto parse pwrctrl active pol from GPIO_ACTIVE_{HIGH,LOW} flag.
    - Reset pmic and soc when system reboot if required.

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: I29e60c2a717cee5c9d1e3c6e46ee687352d4a1be
```

2.3 DTS 配置

DTS 的配置包括：I2C 挂载、主体、regulator等部分。如下以`arch/arm/boot/dts/rk3506-rk801.dts`为例。

硬件设计：



DTS配置：

```
&i2c0 {
    status = "okay";
    ......

    rk801: rk801@27 {
        // 不可修改。
        compatible = "rockchip,rk801";
        status = "okay";
```

```
        reg = <0x27>;

        // PMIC_INT 引脚配置。此处是GPIO0_C2、低电平有效。
        interrupt-parent = <&gpio0>;
        interrupts = <RK_PC2 IRQ_TYPE_LEVEL_LOW>;

        // PWRCTRL 引脚配置。此处是GPIO0_C1、高电平有效。
        pwrctrl-gpios = <&gpio0 RK_PC1 GPIO_ACTIVE_HIGH>;

        // Pinctrl 配置。
        pinctrl-names = "default", "pmic-reset";
        // 初始化pmic_int脚状态。
        pinctrl-0 = <&pmic_int>;
        // 设置reboot时pwrctrl的状态。这样配置后，pmic会跟随SoC一起复位自己的各路电压。
        pinctrl-1 = <&soc_pwrctrl_reset>;

        // 不可修改。声明RK801具备管理系统下电的功能。
        rockchip,system-power-controller;

        // 不可修改。声明PMIC_INT作为唤醒源。
        wakeup-source;

        // 各路regulator的输入源。
        // vcc1~7分别对应: BUCK1/2/3/4, LDO1/2, SWTICH。
        vcc1-supply = <&vcc12v_dc>;
        vcc2-supply = <&vcc12v_dc>;
        vcc3-supply = <&vcc5v0_sys>;
        vcc4-supply = <&vcc5v0_sys>;
        vcc5-supply = <&vcc3v3_sys>;
        vcc6-supply = <&vcc3v3_sys>;
        vcc7-supply = <&vcc3v3_sys>;

        regulators {
            vcc3v3_sys: DCDC_REG1 {
                regulator-name = "vcc3v3_sys";
                regulator-min-microvolt = <3300000>;
                regulator-max-microvolt = <3300000>;
                regulator-initial-mode = <0x1>;
                regulator-boot-on;
                regulator-always-on;
                regulator-state-mem {
                    regulator-mode = <0x2>;
                    regulator-on-in-suspend;
                    regulator-suspend-microvolt = <3300000>;
                };
            };

            vcc5v0_sys: DCDC_REG2 {
                regulator-name = "vcc5v0_sys";
                regulator-min-microvolt = <5000000>;
                regulator-max-microvolt = <5000000>;
                regulator-initial-mode = <0x1>;
                regulator-boot-on;
```

```
            regulator-always-on;
            regulator-state-mem {
                regulator-mode = <0x2>;
                regulator-on-in-suspend;
                regulator-suspend-microvolt = <5000000>;
            };
        };

        vcc_ddr: DCDC_REG3 {
            regulator-name = "vcc_ddr";
            regulator-boot-on;
            regulator-always-on;
            regulator-state-mem {
                regulator-mode = <0x2>;
                regulator-on-in-suspend;
            };
        };

        vdd_cpu: DCDC_REG4 {
            regulator-name = "vdd_cpu";
            regulator-min-microvolt = <500000>;
            regulator-max-microvolt = <1500000>;
            regulator-initial-mode = <0x1>;
            regulator-boot-on;
            regulator-always-on;
            regulator-state-mem {
                regulator-mode = <0x2>;
                regulator-off-in-suspend;
            };
        };

        vdd0v9_sys: LDO_REG1 {
            regulator-name = "vdd0v9_sys";
            regulator-min-microvolt = <900000>;
            regulator-max-microvolt = <900000>;
            regulator-boot-on;
            regulator-always-on;
            regulator-state-mem {
                regulator-on-in-suspend;
                regulator-suspend-microvolt = <900000>;
            };
        };

        vcc_1v8: LDO_REG2 {
            regulator-name = "vcc_1v8";
            regulator-min-microvolt = <1800000>;
            regulator-max-microvolt = <1800000>;
            regulator-boot-on;
            regulator-always-on;
            regulator-state-mem {
                regulator-on-in-suspend;
                regulator-suspend-microvolt = <1800000>;
            };
```

```
            };

            vcc_3v3: SWITCH_REG {
                regulator-name = "vcc_3v3";
                regulator-boot-on;
                regulator-always-on;
                regulator-state-mem {
                    regulator-on-in-suspend;
                };
            };
        };
    };
};

&pinctrl {
    ......

    pmic {
        // pmic_int设置常态为高电平，即低电平有效。
        pmic_int: pmic-int {
            rockchip,pins = <0 RK_PC2 RK_FUNC_GPIO &pcfg_pull_up>;
        };

        // 系统复位时pwrctrl引脚应该保持的电平状态，必须跟SoC上电默认状态相反。
        // 比如：默认上电是down，则配置为output hight；反之则为output low。
        soc_pwrctrl_reset: soc-pwrctrl-reset {
            rockchip,pins = <0 RK_PC1 RK_FUNC_GPIO &pcfg_output_high>;
        };
    };
};
```

regulator 配置项：

- `regulator-compatible`：驱动注册时需要匹配的名字，不能改动，否则会加载失败；
- `regulator-name`：电源的名字，建议和硬件图上保持一致，使用 regulator_get 接口时需要匹配这个名字；
- `regulator-init-microvolt`：u-boot阶段的初始化电压，kernel阶段无效；
- `regulator-min-microvolt`：运行时可以调节的最小电压；
- `regulator-max-microvolt`：运行时可以调节的最大电压；
- `regulator-initial-mode`：运行时 DCDC 的工作模式，一般配置为 1。 1：force pwm，2：auto pwm/pfm；
- `regulator-mode`：休眠时 DCDC 的工作模式，一般配置为 2。1：force pwm， 2：auto pwm/pfm；
- `regulator-boot-on`：存在这个属性时，在注册 regulator 的时候就会使能这路电源；
- `regulator-always-on`：存在这个属性时，表示运行时不允许关闭这路电源且会在注册的时候使能这路电源；
- `regulator-on-in-suspend`：休眠时保持上电状态。想要关闭该路电源则改成”regulator-off-in-suspend”；
- `regulator-suspend-microvolt`：休眠不断电情况下的待机电压。

说明：如果 `regulator-min-microvolt` 和 `regulator-max-microvolt` 的电压相等，则在注册这个 regulator 的时候系统框架默认会把这个电压设置下去并使能这路电源，不需要使用者干预。

如果 `regulator-boot-on` 或者 `regulator-always-on` 存在，则系统框架在注册这路 regulator 的时候默认会进行 enable，此时的这路 regulator 的电压有 2 种情况：如果 `regulator-min-microvolt` 和 `regulator-max-microvolt` 的电压相等，则系统框架会把这路电压设置为当前这个电压值；如果 `regulator-min-microvolt` 和 `regulator-max-microvolt` 的电压不相等，则此时的电压是 PMIC 的本身的硬件默认上电电压。

pwrkey配置项：

rk801节点下如果没有指定`pwrkey`节点则等效于使能。如果确定要关闭，请手动添加节点指定为`disabled`。

```
rk801: rk801@27 {

    ......

    pwrkey {
        status = "disabled";
    };
};
```

2.4 函数接口

如下几个接口基本可以满足日常使用，包括 regulator 开、关、电压设置、电压获取等：

1. 获取 regulator：

   `struct regulator *regulator_get(struct device *dev, const char *id)`

   dev 默认填写 NULL 即可，id 对应 dts 里的 regulator-name 属性。

2. 释放 regulator

   `void regulator_put(struct regulator *regulator)`

3. 打开 regulator

   `int regulator_enable(struct regulator *regulator)`

4. 关闭 regulator

   `int regulator_disable(struct regulator *regulator)`

5. 获取 regulator 电压

   `int regulator_get_voltage(struct regulator *regulator)`

6. 设置 regulator 电压

   `int regulator_set_voltage(struct regulator *regulator, int min_uV, int max_uV)`

   传入的参数时保证 min_uV = max_uV，由调用者保证。

7. 范例

```
struct regulator *rdev_logic;

rdev_logic = regulator_get(NULL, "vdd_logic");          // 获取vdd_logic
regulator_enable(rdev_logic);                           // 使能vdd_logic
regulator_set_voltage(rdev_logic, 1100000, 1100000);    // 设置电压1.1v
regulator_disable(rdev_logic);                          // 关闭vdd_logic
regulator_put(rdev_logic);                              // 释放vdd_logic
```

说明：4.4和更高版本的内核还提供了`devm_`开头的regulator接口帮开发者管理要申请的资源。

3. 调试

因为 PMIC 涉及的驱动在使用逻辑上都不复杂，重点都体现在最后的寄存器设置上。用户可以使用内核自带的`i2cdump`命令，也可以使用如下方式：

节点：`/sys/rk8xx/rk8xx_dbg`

```
# 读寄存器
echo r [addr]  > /sys/rk8xx/rk8xx_dbg
# 写寄存器
echo w [addr] [value] > /sys/rk8xx/rk8xx_dbg
```

使用范例：

```
echo r 0x2f > /sys/rk8xx/rk8xx_dbg          # 读取0x2f寄存器的值
echo w 0x2f 0x9c > /sys/rk8xx/rk8xx_dbg     # 设置0x2f寄存器的值为0x9c
```

> 建议写操作执行完之后最好再读一遍确认是否写成功。